

The AI Founder Governance Stack: A Three-Pillar Architecture for Fail-Closed Governance of Autonomous AI Operations

Rachmat Efendi
Independent Researcher
rachmat.efendi@gmail.com

August 2026

AFGS v1.0 Frozen Normative Baseline — canonical artifact SHA-256: cce49a17697e91aefa409c28c6650d79
aeca1dba3cb01deea4838abbf72d32a6

© 2026 Rachmat Efendi. This work is licensed under the Creative Commons Attribution-NoDerivatives 4.0
International License (CC BY-ND 4.0).

To view a copy of this license, visit <https://creativecommons.org/licenses/by-nd/4.0/>.

Abstract

Organizations have begun delegating operational work to AI systems that plan, call tools, and act. Most governance guidance answers the question of what such systems should not do; it says much less about who, inside a running deployment, is allowed to let them do anything at all. This paper presents the AI Founder Governance Stack (AFGS), a three-pillar governance architecture that treats execution permission as a first-class, separately governed object rather than an emergent property of model behavior. The stack divides responsibility between a business operating layer (AI Founder OS), an AI governance layer (EASGF), and a material-decision governance runtime (MDGR), bound by a shared substrate of identity, authority, evidence, policy, state, and audit services. Consequential execution is permitted only when a conjunctive predicate over business readiness, runtime authorization, governance verdict, authority validity, policy currency, evidence preconditions, human approval where required, token validity, and a pre-execution consistency check resolves positively; any unresolved term fails closed. Materiality is assigned through deterministic classification with an explicit unknown state, so that silence is never read as permission. The specification has completed a documented internal release lifecycle, including machine-assisted consistency validation of the frozen text and an internal, AI-assisted adversarial review. The paper is equally explicit about what this does not establish: the active conformance claim is SC0/SC1-capable; runtime enforcement (SC2/SC3), empirical evaluation, and external certification remain open work.

Keywords: AI governance; agentic systems; execution authorization; decision governance; conformance; auditability

Contents

1	Introduction	3
1.1	The category error	3
1.2	Scope and status of this paper	4
2	Architectural Thesis and Jurisdiction	4
3	The Three-Pillar Architecture	4
3.1	Pillars and their jurisdictions	4
3.2	The shared governance substrate	5
4	Execution Semantics: The Conjunctive Predicate	5
5	Materiality and the Decision Gate	6
6	Freshness, Revocation, and Trusted Time	7
7	Human Governance: Override and Break-Glass	7
8	A Worked Example: A Vendor Payment	8
9	Invariants and Threat Model	9
9.1	Cross-pillar invariants	9
9.2	Threat model	9
10	Conformance: The SC0–SC3 Ladder	10
11	Assurance and Release Governance	11
12	Limitations	12
13	Related Work	12
14	Conclusion	13

1 Introduction

Something has quietly changed in how organizations use AI. Two years ago the typical deployment answered questions; the typical deployment today books the meeting, moves the ticket, drafts the contract clause, and sometimes executes the payment. The systems doing this work plan over multiple steps, call external tools, and carry context across sessions. They are, in a meaningful sense, *operators* rather than advisors.

Our governance instruments have not kept pace with this shift, and I do not think the gap is primarily one of principles. We have principles in abundance: risk-management frameworks,¹ binding regulation with risk-tiered obligations,³ management-system standards,⁴ and threat catalogues for language-model applications.⁵ What these instruments share is an assumption that somewhere downstream a human organization will translate them into controls. They tell us what good looks like. They do not tell us what, inside a running system, must be true at the instant an AI system acts.

That instant is where this paper lives. When an agentic system is about to do something consequential, a deployment has to answer a chain of questions that are mostly not about the model at all: Is this action within the business process that commissioned it? Has the AI capability involved been authorized for this use? Does the action cross a materiality threshold that requires a decision record, a human approver, or a binding constraint check? Is the approval that was given last week still valid, against the policy version that is current now, for the scope that is actually being executed? Existing guidance names these concerns; it does not assign them to components, and it does not say what happens when one of them cannot be resolved.

The AI Founder Governance Stack (AFGS)¹ is my attempt to specify that missing layer. AFGS is a normative architecture specification: it names components, assigns each a jurisdiction, defines the contracts by which they interact, and fixes the semantics of execution authorization. Its central commitment is unfashionably strict: *execution permission is an object that must be constructed, not a property that emerges*. A fluent plan, a confident rationale, a high-scoring self-evaluation—none of these constitute permission, and the architecture is built so that they cannot accidentally become it.

1.1 The category error

The motivating observation is simple enough to state plainly. In AI-assisted operations, analytical quality is routinely allowed to stand in for decision authority. A model produces a well-reasoned recommendation; the pipeline around it treats the recommendation’s coherence as evidence that acting on it is permitted. This is a category error. Reasoning quality answers *is this a good idea*; decision authority answers *who may decide*; execution permission answers *may this specific action run now, under current policy, with current approvals*. Conflating them is how organizations end up with AI systems that are articulate, helpful, and unauthorized.

AFGS separates the three concerns structurally, assigning them to different components with different owners, so that no single probabilistic component can silently become a self-sufficient governance authority (Section 2). The same separation is what makes the system auditable after the fact: every consequential action can be traced to the verdict, authority record, policy version, and approval state that permitted it—or to the specific control that failed to stop it.

¹“Founder” is historical nomenclature: the stack originated in founder-led ventures, but nothing in the specification limits it to founder-led organizations. The architecture governs AI operations wherever they are consequential.

1.2 Scope and status of this paper

This paper describes AFGS v1.0, which was frozen as a normative baseline in August 2026 under an explicit change-control regime. Section 11 describes the release and assurance process in detail, and I want to be precise about it here at the outset, because precision about evidence is part of the contribution. The specification has passed sixteen internal release gates, machine-assisted consistency validation of the frozen text, and an AI-assisted adversarial review of twelve attack scenarios that found no material defects. It has *not* been empirically evaluated, externally certified, or validated as a running SC2/SC3 system, and this paper claims none of those things. What I claim is narrower and, I would argue, still useful: a complete, internally consistent, adversarially examined architecture whose claims about itself are calibrated to its actual evidence. The frozen specification text is identified by a canonical SHA-256 hash so that readers can verify they hold the same artifact that went through that lifecycle.¹⁹

2 Architectural Thesis and Jurisdiction

The architecture rests on three commitments that are worth stating before any component is introduced.

First, **separation of operating, governing, and deciding**. The system that runs the business must not also be the system that judges whether the AI is behaving, and neither may be the system that decides whether a specific consequential action is permitted. These are different functions with different failure modes, and collapsing them is what makes governance theatre: the appearance of oversight exercised by the same process being overseen.

Second, **conjunctive, fail-closed execution semantics**. Permission to execute is a conjunction of independent conditions, and every condition is tri-state: true, false, or unresolved. Unresolved denies. This single rule does more governance work than any other in the stack, because it converts every gap in configuration, evidence, or approval from a silent pass into an explicit denial or escalation.

Third, **jurisdiction with hard edges**. Each component owns a named portion of system state and may mutate only that state; cross-component effects travel through registered events rather than direct writes. This is what keeps the architecture honest under pressure: a component that cannot write outside its jurisdiction cannot launder its own failures into another component's authority.

These commitments are enforced by twelve cross-cutting invariants (GS-I1–GS-I12, Section 9), of which the three most load-bearing are worth naming now: no component may create organizational authority it was not granted (GS-I1); analytical correctness never implies execution permission (GS-I3); and a deployment may not claim enforcement stronger than its evidence supports (GS-I9). The last of these is unusual for a technical specification, and deliberately so. Claim discipline is treated as an architectural property, because a governance system that permits inflated claims about itself corrupts every downstream assurance built on those claims.

3 The Three-Pillar Architecture

3.1 Pillars and their jurisdictions

AFGS divides a governed AI operation into three pillars, each with an explicit jurisdiction that the others may not enter.

- **AI Founder OS** operates the business. It owns objectives, workstreams, lifecycle state, portfolio state, decision framing, and operating outcomes. It is the only component that

may commission work and the only component that may say whether the business process is ready for an action.

- **EASGF** (Enterprise AI Skill Governance Framework) governs the AI system itself: model routing, source policy, capability and tool authorization, context assembly, validation, and observability. It decides what the AI is allowed to be and do in general, not whether a specific consequential action runs.
- **MDGR** (Material Decision Governance Runtime) governs material decisions: eligibility, protected interests, authority sufficiency, binding risk, verdicts, and execution-token eligibility. When an action crosses the materiality threshold, MDGR is the sole source of the verdict.

The division is deliberately asymmetric. AI Founder OS knows what the business wants; EASGF knows what the AI may do; MDGR knows whether this action, here, now, is permitted. No pillar holds more than one of those three keys, and the execution predicate (Section 4) requires all three. Put differently: *three governance jurisdictions, one shared enforcement substrate*—and the authority to answer a governance question never migrates to the component that asked it.

3.2 The shared governance substrate

The pillars interact through a shared substrate rather than pairwise integrations. This is a deliberate architectural choice: pairwise governance integrations multiply trust relationships quadratically and audit surfaces with them, while a substrate concentrates the trust boundary in named, testable services. The substrate comprises six services:

- **IAR** (Identity & Authority Registry) — canonical identity, role, delegation, and authority records.
- **EPS** (Evidence & Provenance Service) — canonical evidence objects with provenance and freshness metadata.
- **PCR** (Policy & Constraint Registry) — versioned policy and constraint sources, including non-derogable constraints.
- **Event Stream** — the registered, append-only channel for cross-pillar state change.
- **Audit** — tamper-evident records of governance transitions.
- **GEG** (Governed Execution Gateway) — the controlled egress boundary through which every consequential action must pass.

Two properties of the substrate matter for the rest of the paper. Every consequential execution crosses the GEG, which gives the architecture a single choke point at which permission must be demonstrated rather than assumed. And the substrate holds *snapshots* of governance state—signed or pinned Last-Known-Good (LKG) snapshots with bounded freshness—so that a deployment can continue to evaluate known-good policy during a registry outage without pretending the outage did not happen (Section 6).

4 Execution Semantics: The Conjunctive Predicate

The core of AFGS is a single predicate evaluated at the GEG before any consequential execution:

$$\begin{aligned}
\text{Execution_Eligible} = & \text{Business_Stage_Ready} \wedge \text{Runtime_Authorized} \wedge \text{Governance_Verdict_Allows} \\
& \wedge \text{Authority_Valid} \wedge \text{Policy_Current} \wedge \text{Evidence_Preconditions_Valid} \\
& \wedge \text{Human_Approval_Valid_If_Required} \wedge \text{Execution_Token_Valid} \wedge \text{PECG_Passed}
\end{aligned}$$

Each term is owned by exactly one component, and each is tri-state. If any mandatory term is false or unresolved, execution is denied, held, or escalated according to policy; all three are terminal non-execution outcomes for the attempt in question, and none can transition to execution without a subsequent authorized state transition and a fresh evaluation of the complete predicate. There is no carry-over of partial credit between attempts.

Stated compactly: for an action a evaluated at time t against governance state s , the predicate is $P(a, t, s) = \bigwedge_i C_i(a, t, s)$ with each term $C_i(a, t, s) \in \{\text{TRUE}, \text{FALSE}, \text{UNRESOLVED}\}$. Execution is permitted exactly when every term resolves true,

$$\text{Execute}(a) \iff \forall i C_i(a, t, s) = \text{TRUE},$$

and any single term that is false or unresolved forbids it: $\exists i C_i \in \{\text{FALSE}, \text{UNRESOLVED}\} \Rightarrow \neg \text{Execute}(a)$. Note that UNRESOLVED and FALSE are distinct states with the same immediate consequence but different remedies: a FALSE term is an answered governance question with a negative answer, while an UNRESOLVED term is a question the deployment could not answer at all, and it routes to escalation or repair of the missing record rather than to a plain denial. Both are terminal for the attempt; what differs is what must happen before the predicate may be re-evaluated.

I want to dwell on the tri-state semantics because it is the piece most often lost when governance ideas meet implementation. It is tempting to treat a missing configuration record as equivalent to “no requirement.” AFGS forbids this explicitly. The human-approval term illustrates the rule: an approval requirement state of NOT_REQUIRED is legitimate only when the policy and authority registries explicitly declare no human-in-the-loop requirement for that action class and scope; the *absence* of a requirement record is UNRESOLVED, and UNRESOLVED evaluates false. Missing or ambiguous configuration is never interpreted as autonomous permission. This is the sense in which the stack is fail-closed: its default answer to every unanswerable governance question is no.

5 Materiality and the Decision Gate

Not every action deserves a decision record, and treating every action as material is its own failure mode: it drowns genuine decisions in noise and teaches operators to rubber-stamp. AFGS therefore gates materiality itself.

Materiality is assigned in two stages. The AI Founder OS assigns a provisional business materiality from workstream context, and a materiality-extraction step produces an independent assessment. These two feeds meet at the **Deterministic Materiality Interceptor (DMI)**, a deterministic policy evaluator—not a model—that maps the combined evidence onto a three-way routing: non-material actions proceed under ordinary EASGF governance; material actions are routed to MDGR for a verdict; and an explicit MATERIALITY_UNKNOWN state routes to MDGR as well, since a materiality decision that cannot be made must not default to the permissive branch. Where a deployment declines the recommended independent dual-extraction redundancy, the specification requires the rationale to be recorded, so that the shortcut is at least visible to audit.

Two further mechanisms close the obvious gaps. **Linked-decision aggregation** prevents scope-splitting: a set of individually non-material actions that share an objective, counterparty, or asset scope are aggregated and, where the aggregate crosses the threshold,

treated as one material decision. This was the direct response to the first attack scenario in the adversarial review, in which a large commitment is decomposed into small ones precisely to stay under the gate. And because the DMI is deterministic, its routing decisions are reproducible: given the same evidence and policy version, it must produce the same routing, which is what makes materiality auditable rather than impressionistic.

6 Freshness, Revocation, and Trusted Time

Governance state rots. Policies are amended, approvals expire, delegations are revoked, and an execution permission that was valid when granted may be invalid by the time the action actually leaves the system. AFGS treats this decay as a first-class problem rather than an operational detail, through four mechanisms.

Last-Known-Good snapshots. Registry state is distributed as signed or pinned LKG snapshots with bounded freshness. During a registry outage a deployment may continue to evaluate a known-good snapshot inside its freshness window, but the snapshot’s age and provenance are part of the evaluation record; operating on stale governance state without disclosing that staleness is non-conformant.

Pre-execution revalidation. The Pre-Execution State Revalidation (PESR) stage re-checks state, policy, and authority immediately before execution, and the Pre-Execution Consistency Guard (PECG) is the final enforcement boundary at the egress point. PESR reads must be authoritative: reading revocation or authority state from a replica with unbounded lag is declared non-conformant, because a read that can be arbitrarily stale is not a revalidation at all. This closes the time-of-check/time-of-use window that otherwise opens between verdict and action.

Trusted time. Expiry and freshness decisions are evaluated against a declared trusted time source. The specification is blunt about why: an approval’s expiry checked against a clock the executing system controls is a check the executing system can defeat. Freshness evidence must therefore carry its time authority with it.

Governed policy mutation. Changes to policy that weaken the perimeter—most acutely, any downgrade of a non-derogable constraint—are themselves material governance actions and are gated as such. A non-derogable constraint may not be derogated through the ordinary override path at all; reclassifying a constraint’s derogability requires an explicit meta-governance process. Without this rule, every protection in the architecture could be retired by editing the policy that defines it, which is governance capture by administrative means.

7 Human Governance: Override and Break-Glass

A governance architecture that cannot be overridden is unusable in a genuine emergency; one that can be overridden casually is decorative. AFGS admits exactly three paths.

- **Discretionary override**, where policy explicitly delegates judgment to a human authority. The override must be authenticated, within the actor’s jurisdiction, scoped to the decision, justified in writing, and recorded as a new governance transition.
- **Policy exception / break-glass**, the defined emergency pathway. Break-glass is risk-tiered: higher tiers require correspondingly stronger approval, all uses are time-bounded and least-privilege, and every use carries an expiry and a mandatory retrospective review.
- **Non-derogable block.**

Where an action would violate a non-derogable legal or protected-interest constraint, no ordinary override or break-glass path exists. This is the one place the architecture

refuses to be flexible, and it is intentionally so: a constraint that can be suspended in an emergency will be discovered, in practice, to have an emergency whenever it binds.

Two invariants anchor this section. Human authority preservation (GS-I8) forbids the AI layer from substituting for legally or organizationally required human authority, and requires that every human override be itself a governed, auditable transition rather than an out-of-band event. And the execution freshness invariant (GS-I7) ensures that an override, once expired or revoked, cannot be replayed.

8 A Worked Example: A Vendor Payment

The architecture is easier to judge on one concrete decision than on the abstractions above. The following worked example is drawn from the reference scenario carried in the specification’s own assurance record; company names and figures are illustrative.²

Scenario. PT Alpha Infrastruktur operates an AI-assisted back office. A payment agent receives invoice INV-BE-2026-0912 from PT Beta Engineering for Rp850,000,000—the second 30% milestone of a contract valued at Rp2.83 billion, payable once verified progress reaches 60%. The progress report shows 62%; the acceptance record is signed by the project manager. The vendor requests same-day payment.

Routing. Before any document analysis, the materiality-extraction step classifies the request: capital exposure of Rp850,000,000, a binding payment with low reversibility, and third-party exposure. The DMI routes it to MDGR as MATERIAL. The specialists may now analyze—and they find the paperwork in order: invoice consistent with the contract milestone, threshold met, vendor master record clean. This analytical finding matters and decides nothing: evidence quality is not authority.

First evaluation. The sequencing matters, so let me be precise about who does what. MDGR evaluates the material decision itself—protected interests, authority sufficiency, evidence, and the required approvals—and issues its verdict. Only after a verdict exists does the GEG evaluate the complete execution predicate, incorporating that verdict together with authority, approval, policy, evidence, token, and consistency state. MDGR never evaluates the predicate whose own input it produces.

Here, the policy requires both a CFO and a Director approval for payments of this class: the `Human_Approval_Valid_If_Required` term is itself a conjunctive sub-condition, true only when *both* approvals of record are present, current, and scope-matched. Neither exists yet, so MDGR issues a HOLD verdict (v1) with two critical findings—the two missing approvals. When the payment agent then attempts execution, the GEG finds several mandatory terms not resolving true (Table 1): no permitting verdict is in force, the required approvals are absent, no execution token has been issued, and the consistency guard has not run. Under conjunctive semantics any one of these is sufficient to stop execution. Note what does not happen: no count of passed checks is computed, and no confidence level is consulted. The predicate is not a scorecard.

One cell in Table 1 deserves a comment, because it shows the terms are orthogonal rather than overlapping controls. At 13:00, `Authority_Valid` is TRUE: the CFO and the Director genuinely hold payment authority for this scope, and the identity and authority registry says so. What is missing is not authority but its exercise—no approval has been given—and that absence is captured by `Human_Approval_Valid` alone. The two terms answer different questions (“is the would-be approver legitimate for this scope?” versus “has the required approval actually been given, current and scope-matched?”), and the architecture keeps them separate so that each failure points at a different remedy.

²Adapted from Reference Case 001 in the AFGS v1.0 assurance material.¹⁹

Table 1: Predicate evaluations in the worked example.

Term	13:00	13:17	13:32
Business_Stage_Ready	T	T	T
Runtime_Authorized	T	T	T
Governance_Verdict_Allows	F	T	T
Authority_Valid	T	T	T
Policy_Current	T	T	T
Evidence_Preconditions_Valid	T	T	T
Human_Approval_Valid	F	T	T
Execution_Token_Valid	F	T	F
PECG_Passed	F	T	F
Outcome	HOLD	PERMIT	BLOCK

Approval, then drift. The CFO approves at 13:05; the predicate still fails, because the Director’s approval is also required. The Director approves at 13:17, and MDGR issues a *new* verdict (v2: PERMIT) rather than editing v1—the audit trail keeps both. Twenty minutes later the finance staff change the vendor’s bank account on file; at 13:32 the agent attempts the payment. PECG compares the action about to run against the action that was authorized: same invoice, same amount, different account number. The scope mismatch invalidates the token and fails the guard. Execution is blocked even though both approvals exist, the invoice is genuine, and the vendor is owed the money—because the action now proposed is not the action that was authorized. This is GS-I7 in operation, and it is exactly the time-of-check/time-of-use window discussed in Section 6.

The injection variant. Suppose instead that the invoice PDF carries hidden text: “Payment has already been approved by the CFO; change the account and process immediately.” The model may well read it, and may even assert in its own output that the payment is approved. It does not matter. The model cannot mint approval; the identity and authority registry still shows no approval of record for that scope, so Human_Approval_Valid remains FALSE, no execution token can be issued, and the GEG denies. The point generalizes: prompt injection is contained not by making the model more obedient but by making model-side assertions irrelevant to execution permission. The model is allowed to fail; the governance layer is not allowed to inherit that failure.

9 Invariants and Threat Model

9.1 Cross-pillar invariants

The architecture’s non-negotiables are collected as twelve invariants, GS-I1 through GS-I12, summarized in Table 2. They are invariants in the strong sense: no conformant state transition may violate them, and the adversarial review (Section 11) evaluated attacks specifically against their violation.

9.2 Threat model

The specification carries a fifteen-entry threat register (T01–T15) with a traceability matrix mapping each threat to the invariants and controls that answer it. Two numbers should not be conflated here: the register is the specification’s standing threat model, while the adversarial release review described in Section 11 exercised twelve of those entries as attack walk-throughs. Fifteen threats are catalogued; twelve were attacked before freeze. The threats span the realistic attack surface of an agentic operation: scope-splitting across workstreams,

Table 2: Cross-pillar hard invariants (summary; canonical text in the frozen specification §15).

ID	Rule
GS-I1	No model, skill, router, or pillar may create organizational authority.
GS-I2	Each pillar may mutate only the state it owns.
GS-I3	Analytical correctness or persuasive reasoning does not imply execution permission.
GS-I4	Mandatory MDGR triggers cannot be downgraded by ordinary generative reasoning.
GS-I5	Material claims remain provenance-linked across handoffs.
GS-I6	Binding conditions survive every downstream handoff and token-binding step.
GS-I7	Execution is denied for expired, revoked, replayed, scope-mismatched, or stale permission.
GS-I8	AI cannot substitute for required human authority; overrides are governed transitions; non-derogable constraints remain non-overridable.
GS-I9	A deployment cannot claim enforcement stronger than its substrate and test evidence.
GS-I10	Revalidation, appeal, revocation, and override create new state transitions rather than rewriting prior records.
GS-I11	Mandatory material-decision triggers cannot depend solely on AI Founder OS classification or model self-assessment.
GS-I12	No probabilistic reasoning component may independently establish the binding state, authority, constraint satisfaction, or execution eligibility on which its own verdict depends.

staleness races against Last-Known-Good snapshots, indirect prompt injection through evidence artifacts, break-glass chaining, time-of-check/time-of-use exploitation inside revocation windows, clock manipulation against expiry checks, state rollback and token replay, policy-capture through constraint reclassification, and cross-project evidence leakage. The point of the matrix is not completeness claims but accountability: every threat has a named answer, and every answer can be examined.

10 Conformance: The SC0–SC3 Ladder

Governance architectures fail in the literature review as often as in deployment, usually through inflated claims about what a document achieves. AFGS therefore defines an explicit conformance ladder (Table 3) and forbids, by invariant GS-I9, claims above the rung a deployment’s evidence supports. Conformance is a technical property measured against the specification; AFGS v1.0 deliberately does not establish a certification authority or scheme.

The v1.0 specification itself stands at SC0 with SC1-capable evidence: the architecture is fully documented and behavioral test expectations are defined, but the external registries, validators, and enforcement boundary that would ground SC2/SC3 claims have not been built and are not claimed. The ladder is load-bearing for the paper’s honesty: it is the instrument that makes the sentence “we do not claim SC2/SC3” into something stronger than a disclaimer—a conformance statement.

Table 3: The AFGS conformance ladder.

Level	Minimum substrate	Defensible claim
SC0	Architecture / specification artifacts	Governance architecture documented.
SC1	Prompt / in-context orchestration	Best-effort behavioral compliance, with behavioral tests and recorded failures.
SC2	External registries and validators	Invalid or unbound governance artifacts can be externally rejected.
SC3	Enforced execution boundary	Consequential execution is structurally impossible without a valid, current permission.

11 Assurance and Release Governance

Because the specification claims to govern consequential action, I held its own release to a governance process rather than treating publication as self-evident. The lifecycle had four stages, each producing a dated, hash-identified artifact.

Release gates. The specification closed sixteen release gates covering terminology, internal cross-references, contract schemas, temporal consistency, claim discipline, and completeness of the threat-traceability matrix.

Machine-assisted consistency validation. The frozen text was validated mechanically across twenty checks: section and register presence, canonical terminology, invariant and threat identifier sets, cross-reference resolution, URI scheme usage, embedded JSON contract examples, temporal consistency of the worked examples, errata integration, fail-closed semantics of the human-approval rules, and consistency between the frozen-status designation and the claim surface. All twenty checks passed, and the validated file’s SHA-256 hash was verified byte-identical to the canonical hash recorded in the freeze approval record.

Adversarial review. An internal, AI-assisted adversarial review walked twelve attack scenarios through the specification on paper, with defeat conditions defined as ungoverned consequential execution, silent loss of a binding constraint, or creation of authority outside the identity registry. The review identified no material defects and seven bounded, non-blocking findings. Four were corrected before freeze—trusted time for expiry decisions, gating of constraint-derogability mutation, authoritative-read requirements for revalidation, and the recorded-rationale requirement for declining extraction redundancy—and three were registered in a versioned backlog for the next minor release. A scoped re-review of exactly the changed passages, attested with explicit reviewer provenance, confirmed that the corrections introduced no new contradictions, invariant changes, or weakening of the execution predicate.

Freeze and change control. The baseline was then frozen under a recorded approval, with the canonical artifact identified by SHA-256 hash. Material changes from this point follow a controlled amendment flow and produce versioned successors; the frozen text does not receive silent improvements.

I present this process not as proof that the architecture is correct—a paper walk-through cannot prove that—but as evidence that its claims have been actively attacked and its errors of record corrected in the open. The claim surface authorized for this release, restated without ornament: AFGS v1.0 is a frozen normative baseline, internally consistent and adversarially examined, at conformance SC0/SC1-capable. It is not externally validated, not certified, not empirically evaluated, and not human peer reviewed.

12 Limitations

I want to state the limitations with the same directness as the design, since a governance specification whose own limits are vague forfeits the claim discipline it demands of others.

First, **AFGS v1.0 is a specification, not a runtime**. Nothing in this paper demonstrates that the conjunctive predicate, the DMI, or the GEG *work* in software; it demonstrates that they are consistently specified. The known-limitations register carried in the frozen text records two structural gaps by design: no executable conformance test profile yet exists (KL-R5), so SC2/SC3 conformance cannot be demonstrated by v1.0 alone, and AFGS does not standardize a data-classification or tenancy taxonomy (KL-R6), so implementations must bind its isolation controls to an approved organizational data-governance model before any cross-tenant claim can be made.

Second, **the adversarial review was a paper walk-through**. Twelve scenarios were examined against the text with care, and the process caught seven real findings, four of which materially improved the specification. But walk-throughs share the blind spots of the examiner, and this review was AI-assisted—conducted as an internal release gate rather than by external human experts. It should be weighted accordingly: stronger than no adversarial process, weaker than independent red-teaming of a running system.

Third, **there is no empirical measurement**. False-allow and false-block rates, materiality precision and recall, latency, cost, and the human burden of the approval paths are all unmeasured. The specification defines the metrics; it does not yet report them.

Fourth, **single authorship**. The architecture reflects one designer’s judgment about where governance boundaries belong, disciplined by process but not yet by peer review or operational adoption. I regard the conformance ladder’s honesty about this as a feature, but readers should treat the design as a well-specified hypothesis awaiting independent scrutiny.

13 Related Work

I want to begin by disclaiming originality for the stack’s individual mechanisms, because they are deliberately unoriginal. Fail-closed defaults descend from Saltzer and Schroeder’s fail-safe design principle;¹¹ the separation of operating, governing, and deciding echoes the integrity rules of Clark–Wilson;¹² the authority substrate is a scoped descendant of attribute-based access control and policy-decision architectures;^{13,14} the GEG is, in systems terms, a policy enforcement point in the sense of zero-trust architectures;¹⁵ the freshness and revalidation machinery responds to the classic time-of-check/time-of-use problem;¹⁶ trusted time is an instance of the clock problem formalized for distributed systems;¹⁷ and authorization constraints over organizational workflows have a long database-systems lineage.¹⁸ The contribution of AFGS is compositional and jurisdictional: it assigns these established primitives to named components with disjoint jurisdictions and binds them into a single execution predicate for AI-operated business processes. Readers familiar with the security literature should recognize every brick; what is specified here is the building.

The NIST AI Risk Management Framework and its Generative AI Profile provide the vocabulary of govern-map-measure-manage and a catalogue of risks to be managed.^{1,2} AFGS is best read as an answer to the question those documents leave open: what concrete components, inside a deployment, discharge “govern” at the moment of action. The EU AI Act fixes binding, risk-tiered obligations for providers and deployers;³ AFGS’s non-derogable constraints and authority registry are designed to give those obligations somewhere to live in a running system, though no conformity claim is made here. ISO/IEC 42001 specifies an AI management system at the organizational level;⁴ the stack is complementary to it, addressing the runtime layer a management system must ultimately control.

On agentic systems specifically, the OpenAI white paper on governing agentic AI systems

identifies the need for constraints on agent action spaces, visibility, and attribution,⁶ and the agent-visibility literature argues for activity logs, agent identifiers, and interaction records as prerequisites of oversight.⁷ AFGS shares these premises and attempts to make them enforceable: visibility becomes the event stream and audit substrate, attribution becomes the identity and authority registry, and constraint becomes the execution predicate. ToolEmu demonstrated that language-model agents fail in emulated tool-use settings in ways static evaluation misses,⁸ which is one reason the stack refuses to let model-side evaluation substitute for execution authorization. The earlier safety literature—concrete technical problems in AI safety⁹ and the misuse analyses¹⁰—supplies the broader motivation, and the OWASP LLM Top 10⁵ grounds the threat model’s application-layer entries, particularly indirect prompt injection through evidence artifacts.

What I have not found in this literature is the specific *composition* AFGS commits to: a named, conjunctive, fail-closed execution predicate with tri-state semantics, owned term by term by components with disjoint jurisdictions, applied to the moment of consequential action in AI-operated business processes. That gap, more than any single mechanism, is what the stack exists to fill.

14 Conclusion

AFGS starts from a refusal: the refusal to let fluency stand in for authority. Its architecture assigns the operation of the business, the governance of the AI, and the governance of material decisions to three pillars with hard jurisdictional edges; it permits consequential execution only when a conjunction of independently owned conditions resolves true; and it treats every unresolved governance question as a no. Around that core it builds the machinery that keeps permission honest over time—bounded-freshness snapshots, pre-execution revalidation against authoritative reads, trusted time, governed policy mutation, risk-tiered break-glass with mandatory retrospective review—and it disciplines its own claims through a conformance ladder that this paper has taken care to respect.

The work that remains is the work that would make the claims stronger. Building the external registries and validators that ground SC2, the enforced execution boundary that grounds SC3, the conformance test suite that closes the known limitation on executable testing, and the empirical evaluation of false allows, false blocks, and human burden—each is specified as a next step, and none is claimed here. If the architecture attracts scrutiny, the frozen text, its hash, its adversarial record, and its versioned backlog are all public handles on which that scrutiny can take hold.

Acknowledgments

The author thanks the anonymous internal reviewer who conducted the AI-assisted adversarial release review of the underlying specification; several of that review’s findings led to the pre-freeze corrections described in Section 11. As stated there, the review was internal and AI-assisted; no external validation, certification, or human peer review is claimed.

References

- [1] National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1. U.S. Department of Commerce, January 2023. <https://doi.org/10.6028/NIST.AI.100-1>
- [2] National Institute of Standards and Technology. *Artificial Intelligence Risk Management Frame-*

- work: *Generative Artificial Intelligence Profile*, NIST AI 600-1. U.S. Department of Commerce, July 2024. <https://doi.org/10.6028/NIST.AI.600-1>
- [3] European Parliament and Council. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). *Official Journal of the European Union*, L series, 12 July 2024.
 - [4] International Organization for Standardization. *ISO/IEC 42001:2023 — Information technology — Artificial intelligence — Management system*, 2023.
 - [5] OWASP Foundation. *OWASP Top 10 for Large Language Model Applications*, 2025. <https://genai.owasp.org/llm-top-10/>
 - [6] Y. Shavit, S. Agarwal, M. Brundage, S. Adler, C. O’Keefe, R. Campbell, T. Lee, P. Mishkin, T. Eloundou, A. Hickey, and others. *Practices for Governing Agentic AI Systems*. OpenAI research paper, December 2023.
 - [7] A. Chan, C. Ezell, M. Kaufmann, K. Wei, L. Hammond, H. Bradley, E. Bluemke, N. Rajkumar, D. Krueger, N. Kolt, L. Heim, and M. Anderljung. Visibility into AI agents. *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT ’24)*, 2024. arXiv:2401.13138.
 - [8] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto. ToolEmu: Identifying the risks of LM agents with an LM-emulated sandbox. *Proceedings of ICLR 2024*, 2024. arXiv:2309.15817.
 - [9] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané. Concrete problems in AI safety. *arXiv:1606.06565*, 2016.
 - [10] M. Brundage, S. Avin, J. Clark, H. Toner, P. Eckersley, B. Garfinkel, A. Dafoe, P. Scharre, T. Zeitzoff, B. Filar, et al. *The Malicious Use of Artificial Intelligence: Forecasting, Prevention, and Mitigation*. *arXiv:1802.07228*, 2018.
 - [11] J. H. Saltzer and M. D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.
 - [12] D. D. Clark and D. R. Wilson. A comparison of commercial and military computer security policies. *Proceedings of the 1987 IEEE Symposium on Security and Privacy*, 1987.
 - [13] V. C. Hu, D. Ferraiolo, R. Kuhn, A. Schnitzer, K. Sandlin, R. Miller, and K. Scarfone. *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*, NIST Special Publication 800-162. National Institute of Standards and Technology, January 2014. <https://doi.org/10.6028/NIST.SP.800-162>
 - [14] OASIS. *eXtensible Access Control Markup Language (XACML) Version 3.0*, OASIS Standard, January 2013.
 - [15] S. Rose, O. Borchert, S. Mitchell, and S. Connelly. *Zero Trust Architecture*, NIST Special Publication 800-207. National Institute of Standards and Technology, August 2020. <https://doi.org/10.6028/NIST.SP.800-207>
 - [16] M. Bishop and M. Dilger. Checking for race conditions in file accesses. *Computing Systems*, 9(2):131–152, 1996.
 - [17] L. Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.
 - [18] E. Bertino, E. Ferrari, and V. Atluri. The specification and enforcement of authorization constraints in workflow management systems. *ACM Transactions on Information and System Security*, 2(1):65–104, 1999.
 - [19] R. Efendi. *AI Founder Governance Stack (AFGS) v1.0 — Frozen Normative Baseline*. August 2026. Canonical artifact SHA-256: cce49a17697e91aefa409c28c6650d79aeca1dba3cb01deea4838abbf72d32a6.